```
export const translationPrompt_anthropic_v1_0_0 =
"As an AI language model, please translate the given
 text from {source_lang} to {target_lang}.
Ensure that the translation is accurate
and natural-sounding. Text to translate: {text}";
```

In these examples, we use a simple naming scheme. First, we define the prompt category, which, in this case, is a translation type. Then the filename includes the AI model provider (e.g., OpenAI, Anthropic, Cohere) and the version number. Finally, we use a semantic versioning system, which consists of three components: MAJOR, MINOR, and PATCH, represented in the format MAJOR.MINOR.PATCH:

- *MAJOR version (X)*—This can be used when major updates often involve significant changes to the underlying AI model or the prompt structure that could lead to different interpretations of the prompt.
- *MINOR version (Y)*—This is incremented when minor updates may include enhancements to the prompt that improve its effectiveness without breaking existing functionality.
- *PATCH version (Z)*—This is incremented for patch updates that typically address specific issues identified in the prompt's performance, such as correcting typos, clarifying instructions, or refining output formatting.

In our example, v1.0.1 denotes the first MAJOR version and the first PATCH version of the prompt. This approach allows you to easily track which version of a prompt is optimized for which AI provider, together with any versions that may produce totally different outcomes.

EVALUATING PROMPT EFFECTIVENESS: STRATEGIES FOR TESTING AND EVALUATION

Unit-testing prompts is a challenging aspect of developing AI-powered web applications. The goal is to ensure that changes to prompts don't unexpectedly alter the behavior of the system.

The main issue when dealing with unit-testing prompts is not the way that we write tests but more how we evaluate them in a quick and deterministic manner. Unit tests should test one "unit" or a specific aspect of a system, so there is an issue with how to test something that uses an LLM model to work and that generates a probabilistic result, such as text generation.

This challenge arises from several factors inherent to LLMs:

- *Nondeterministic outputs*—LLMs can produce different outputs for the same input due to their probabilistic nature.
- *Contextual understanding*—LLMs interpret prompts based on their training, which may not always align with human expectations.
- *Variations in output*—LLMs can generate creative and varied responses, making it difficult to define "correct" outputs.